NAME: K. SWARNA VARSHINI

REG.NO: 192324081

COURSE CODE: CSA0920

GITHUB LINK:

https://github.com/kswarna1/CSA0920-JAVA.git

PART A–PERSISTENCE DESIGN & PSEUDO CODE

TASK A1–DESIGN A FILE FORMAT

Aim:

To design a delimited text file and a binary file format for storing university bus-pass records and develop operations to add, search, and list pass records.

Theory:

A file is used to store data permanently so that it remains available even after the program terminates.

Two file formats can be used:

- **Delimited text file:** Stores data as readable text, with commas separating fields.
- **Binary file:** Stores data in binary form using DataOutputStream, making it more compact for fixed-size primitive values.

A1.1 – Represent the Record

A. Delimited text file

Each record contains seven fields separated by commas.

Field order	Field name	Example
1	Pass ID	P1001
2	Register Number	192311001
3	Student Name	Arun
4	Route Number	12
5	Boarding Stop	Central
6	Validity Date	2027-03-31

7	Amount Paid	4500.00

Example file content:

P1001,192311001,Arun,12,Central,2027-03-31,4500.00

P1002,192311002,Divya,7,College,2027-03-31,3800.00

If a student's name contains a comma, such as Arun, Kumar, the comma must be escaped or the name must be quoted using a defined CSV format. A CSV parser should be used instead of blindly splitting on commas.

B. Binary file using DataOutputStream

Fields are written in this order:

Field	**Data type**	**Storage**
Pass ID	String	writeUTF()
Register Number	String	writeUTF()
Student Name	String	writeUTF()
Route Number	int	writeInt()
Boarding Stop	String	writeUTF()
Validity Date	String	writeUTF()
Amount Paid	double	writeDouble()

Example Java statements:

```
DataOutputStream dos = new DataOutputStream(
    new FileOutputStream("passes.dat"));
```

dos.writeUTF("P1001");

dos.writeUTF("192311001");

dos.writeUTF("Arun");

dos.writeInt(12);

dos.writeUTF("Central");

dos.writeUTF("2027-03-31");

dos.writeDouble(4500.00);

dos.close();

writeUTF() stores a length-prefixed string, so commas inside names do not cause field-separation problems.

3. A1.2 – Design the Operations

We use BufferedReader and BufferedWriter because they make text-file reading and writing more efficient.

Algorithm 1: Add a New Pass

Algorithm AddPass

Begin

Input passID, registerNo, name

Input routeNo, boardingStop, validityDate, amount

Open file using FileWriter in append mode

Wrap FileWriter with BufferedWriter

Write all seven fields separated by commas

Write a new line

Close BufferedWriter

Display "Pass added successfully"

End

Algorithm 2: Find a Pass by Pass ID

```
Algorithm FindPass
Begin
    Input searchID
    Open file using FileReader
    Wrap FileReader with BufferedReader
    While a line is available
        Read one line
        Split the line into fields
        If number of fields is not 7
            Skip the damaged line
        Else if first field equals searchID
            Display the pass details
            Close reader
            Return
        End If
    End While
    Display "Pass not found"
    Close reader
End
```

Algorithm 3: List Passes by Route Number

```
Algorithm ListByRoute
Begin
    Input requiredRoute
```

Open file using FileReader

Wrap FileReader with BufferedReader

While a line is available

Read one line

Split the line into fields

If number of fields is 7

If route number equals requiredRoute

Display the pass details

End If

Else

Skip the damaged line

End If

End While

Close reader

End

Stream selection:

Class	**Purpose**
FileReader	Reads character data
FileWriter	Writes character data
BufferedReader	Reads text efficiently, line by line
BufferedWriter	Writes text efficiently

FileInputStream	Reads raw bytes
FileOutputStream	Writes raw bytes

For the text-file design, BufferedReader and BufferedWriter are appropriate. FileInputStream and FileOutputStream are more suitable for binary data.

4. A1.3 – Design for Damaged Data

Three problems may occur:

1. **Damaged line:** Check whether the line contains exactly seven fields and validate numeric values such as route number and amount. Skip invalid records and continue reading.
2. **Missing file:** Check whether the file exists using File.exists(). If it does not exist, display a message or create a new file when adding the first pass.
3. **Incomplete write due to power failure:** Write new records to a temporary file, flush and close it, then replace the original file only after the write succeeds. This reduces the risk of losing the complete file. A backup can also be maintained.

5. Alternative Concept and Complexity Analysis

If RandomAccessFile with fixed-length records is used, each record occupies the same number of bytes.

The location of record i is calculated using:

$$\text{Position} = i \times \text{RecordSize}$$

Example:

```
RandomAccessFile file =
    new RandomAccessFile("passes.dat", "rw");
int recordSize = 200;
int recordIndex = 5;
```

file.seek((long) recordIndex * recordSize);

Fixed-size fields must be padded to their allocated lengths. If a name needs 50 bytes but uses only 10 bytes, the remaining 40 bytes are wasted.

Complexity comparison

Operation	**Text file**	**Fixed-length random access**
Find by ID	O(n)	O(1) if record index is known
Add record	O(1) append	O(1) append
List by route	O(n)	O(n)
Storage	Variable size	Fixed size with possible padding

Important: Random access gives O(1) lookup only when the record position is already known. Finding a pass by ID without an index still requires O(n) scanning. An index can be maintained to map pass IDs to record positions.

TASK A2–DON'T READ WHAT YOU DON'T NEED

1. Aim

To analyse the performance of reading a large file one byte at a time and redesign the program using buffered reading to improve efficiency.

2. Theory

Java provides FileInputStream to read data from files byte by byte. However, reading a large file one byte at a time creates many read operations.

Buffering improves performance by reading a group of bytes into memory at once instead of requesting every byte separately.

3. A2.1 – Predict the Cost

Given:

- File size = 50 MB

- Reading method = FileInputStream.read()
- Buffering = Not used

Assume 1 MB = 1,048,576 bytes.

$$50 \times 1{,}048{,}576 = 52{,}428{,}800 \text{ bytes}$$

Therefore, the program makes approximately **52,428,800 read() calls** to read the entire file.

Reason:

- Each read() reads only one byte.
- A separate method call is required for every byte.
- This increases method-call and I/O overhead.
- Therefore, the program becomes slow.

4. A2.2 – Redesign Using Buffering

We can use BufferedInputStream to read multiple bytes at once.

Java Program:

```
import java.io.*;

public class BufferedLineCounter {

    public static void main(String[] args) throws IOException {

        FileInputStream fis = new FileInputStream("attendance.txt");

        BufferedInputStream bis = new BufferedInputStream(fis);

        int count = 0;

        int data;

        while ((data = bis.read()) != -1) {

            if (data == '\n') {

                count++;

            }
```

```
        }
        bis.close();
        System.out.println("Total lines: " + count);
    }
}
```

Explanation:

1. FileInputStream opens the file.
2. BufferedInputStream reads blocks of bytes into an internal buffer.
3. The loop checks each byte for the newline character \n.
4. The line count is increased whenever a newline is found.
5. The stream is closed after reading.

How buffering improves performance:

Assume a buffer size of 8 KB.

$$\text{Buffer size} = 8 \times 1024 = 8192 \text{ bytes}$$

$$\text{Read operations} \approx \frac{52{,}428{,}800}{8192} = 6400$$

Method	**Approximate underlying file reads**
Unbuffered	52,428,800
Buffered (8 KB)	6,400

Buffering reduces the number of underlying file reads significantly. Note that the loop still makes one Java read() call per byte; the buffer reduces actual lower-level file I/O operations.

5. A2.3 – When Buffering Does Not Help

Buffering gives little or no benefit when:

- The file is very small and can be read quickly.
- The data is already available in memory.
- The application performs random access instead of sequential reading.
- The source itself provides data in large blocks.

For example, reading a small 100-byte file may not show a noticeable performance improvement.

6. Alternative Concept and Complexity Analysis

If the entire file is read using Files.readAllLines(), all lines are loaded into a Java list in memory.

Alternative Java code:

```
import java.nio.file.*;
import java.util.*;
public class ReadAllLines {
    public static void main(String[] args) throws Exception {
        List<String> lines =
            Files.readAllLines(Path.of("attendance.txt"));
        System.out.println("Total lines: " + lines.size());
    }
}
```

Comparison

Factor	**Buffered reading**	**Files.readAllLines()**
Reading method	Reads data in blocks	Loads all lines

Time complexity	O(n)	O(n)
Extra memory	O(buffer size)	O(file size)
Large files	More suitable	May consume large memory
Processing	Line/byte by line/byte	All lines available together

Here, $n$represents the file size.

For a 50 MB file:

- Buffered reading requires only a small buffer, such as 8 KB, plus a small amount of program memory.
- Files.readAllLines() requires memory for the complete text, the list, and individual string objects, so actual memory use can be considerably greater than 50 MB.

Decision: Buffered reading is better for processing large files because it uses less memory. Files.readAllLines() is suitable for small files when all lines need to be accessed together.

TASK A3–DISCOVER THE DATA-ACCESS DESIGN PATTERN

1. Aim

To identify repeated database operations in Java classes and design a generic Data Access Object (DAO) interface to reduce code duplication and improve maintainability.

2. Theory

The **DAO (Data Access Object) design pattern** separates database operations from business logic.

It provides a common structure for performing database operations such as:

- Insert
- Find by ID
- Find all
- Update

- Delete

A generic DAO interface can be reused for different entities such as Student, Course, and Marks.

3. A3.1 – Identify the Common Pattern

The three classes StudentDB, CourseDB, and MarksDB perform almost identical database operations.

Repeated steps:

1. Open a database connection.
2. Create an SQL query.
3. Execute the SQL statement.
4. Read the ResultSet.
5. Convert database records into Java objects.
6. Close the database connection.

Differences:

Class	**Table**	**Data**
StudentDB	student	Student details
CourseDB	course	Course details
MarksDB	marks	Student marks

Analysis:

The connection handling, query execution, and result processing are common. Only the table names, SQL queries, and object fields differ.

Therefore, a generic DAO interface can reduce code duplication.

4. A3.2 – Design a Generic Data-Access Interface

A generic interface uses type parameters to support different Java object types.

Generic DAO interface:

```
public interface Dao<T, K> {
    void insert(T obj);
    T findById(K id);
    List<T> findAll();
    void update(T obj);
    void delete(K id);
}
```

Here:

- T represents the entity type.
- K represents the primary key type.

Student class:

```
public class Student {
    int studentId;
    String name;
    String department;
    public Student(int studentId, String name,
                String department) {
        this.studentId = studentId;
        this.name = name;
        this.department = department;
    }
}
```

StudentDao implementation:

```
import java.sql.*;
import java.util.*;
public class StudentDao implements Dao<Student, Integer> {
    Connection con;
    public StudentDao(Connection con) {
        this.con = con;
    }
    public void insert(Student s) {
        try {
            String sql = "INSERT INTO student VALUES(?,?,?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, s.studentId);
            ps.setString(2, s.name);
            ps.setString(3, s.department);
            ps.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    public Student findById(Integer id) {
        try {
            String sql = "SELECT * FROM student WHERE student_id=?";
            PreparedStatement ps = con.prepareStatement(sql);
```

```
            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return new Student(
                    rs.getInt("student_id"),
                    rs.getString("name"),
                    rs.getString("department")
                );
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
    public List<Student> findAll() {
        return new ArrayList<>();
    }
    public void update(Student s) {
        // Update student details
    }
    public void delete(Integer id) {
        // Delete student by ID
    }
}
```

Note: The findAll(), update() and delete() methods above are placeholders to demonstrate the interface structure. A complete implementation must contain their SQL operations.

5. A3.3 – Map Objects to Tables

A Java Student object can be mapped to a relational database table.

Student table:

Column name	Data type	Description
student_id	INT	Primary key
name	VARCHAR(100)	Student name
department	VARCHAR(50)	Department name

SQL table creation:

```
CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50)
);
```

Object-to-table mapping:

Java field	Database column
studentId	student_id
name	name
department	department

The studentId field maps to the primary key student_id, which uniquely identifies each student.

6. Alternative Concept and Complexity Analysis

If SQL queries are written directly inside service or GUI classes, database logic becomes mixed with application logic.

For example:

String sql = "SELECT * FROM student WHERE student_id=?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 101);

ResultSet rs = ps.executeQuery();

If the student table column is renamed, every class containing that SQL query may need to be changed.

With the DAO pattern, database queries are maintained in one dedicated class, making updates easier.

Comparison

Factor	**DAO pattern**	**SQL inside service/GUI**
Code organization	Separate	Mixed
Code duplication	Less	More
Maintenance	Easier	Difficult
Column rename	Mainly DAO changes	Multiple classes may change
Time complexity	Same	Same
Space complexity	Same asymptotic cost	Same asymptotic cost

Complexity analysis:

- Searching a student using a primary-key index: approximately $O(1)$average for a hash index or $O(\log n)$for a B-tree index.
- Retrieving all students: $O(n)$.
- Additional space for a returned list of students: $O(n)$.

The DAO pattern does not directly improve the execution time or memory complexity of SQL operations. Its main advantage is better organization, reusability, and maintainability.

TASK A4–WHAT SHOULD BE SERIALIZED?

1. Aim

To analyse Java object serialization in a hostel management system and identify which fields should be serialized, which should be marked transient, and how serialization handles changes in a class.

2. Theory

Serialization is the process of converting a Java object into a byte stream so that it can be stored in a file or transferred over a network.

Deserialization is the process of converting the stored byte stream back into a Java object.

Java uses the Serializable interface to support object serialization.

3. A4.1 – Decide What to Persist

The hostel management program contains three classes:

- Hostel – contains a list of Room objects.
- Room – contains a list of Resident objects.
- Resident – contains name, register number, password, and database Connection.

Fields to serialize:

Field	**Serialization decision**	**Reason**
Hostel rooms	Serialize	Stores all room details

Room residents	Serialize	Stores resident details
Resident name	Serialize	Required for identification
Register number	Serialize	Identifies the student
Password	Do not serialize	Sensitive information
Database Connection	Do not serialize	Live connection cannot be meaningfully restored from a file

The password should be excluded from this saved object graph and handled securely through a separate credential-management system. A live database connection must be recreated after loading the saved data.

Java example:

```
import java.io.*;
import java.util.*;
class Hostel implements Serializable {
    ArrayList<Room> rooms;
}
class Room implements Serializable {
    ArrayList<Resident> residents;
}
class Resident implements Serializable {
    String name;
    String registerNumber;
    transient String password;
    transient Connection connection;
```

}

The Connection field above requires:

import java.sql.Connection;

What happens when Hostel is serialized?

1. The Hostel object is converted into a byte stream.
2. Its Room objects are serialized.
3. The Resident objects inside each Room are also serialized.
4. Fields marked transient are excluded from default serialization.
5. After deserialization, transient fields receive default values, such as null for object references.

Therefore, the complete object graph is saved except for transient fields.

4. A4.2 – Plan for Change

Suppose a new field called phoneNumber is added to the Resident class after serialization files have already been created.

Example:

```
class Resident implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    String registerNumber;
    String phoneNumber;
}
```

What happens when an old file is read?

- If the old serialized class and new class have compatible serialVersionUID values, the old file can generally be deserialized.

- The newly added phoneNumber field receives its default value, which is null for a String.
- If the serial version IDs are incompatible, Java throws InvalidClassException.

Role of serialVersionUID:

serialVersionUID is a version identifier used during serialization and deserialization to check class compatibility.

Example:

private static final long serialVersionUID = 1L;

Using an explicitly declared version ID helps control compatibility when the class changes.

5. Alternative Concept and Complexity Analysis

Instead of Java serialization, hostel information can be stored in CSV or JSON format.

CSV example:

Name,RegisterNumber,PhoneNumber

Arun,192311001,9876543210

Divya,192311002,9876543211

JSON example:

```
{
  "name": "Arun",
  "registerNumber": "192311001",
  "phoneNumber": "9876543210"
}
```

Comparison

Factor	**Java Serialization**	**CSV / JSON**
Format	Binary	Text

Readability	Not human-readable	Human-readable
File size	Often compact, but depends on object graph	Depends on formatting and data
Save time	O(n)	O(n)
Load time	O(n)	O(n)
Adding new fields	Requires version compatibility handling	Can add optional fields more easily
Compatibility	Java-specific	Supported by many languages

Here, $n$represents the number of residents.

Space complexity:

- Java serialization: $O(n)$to store the complete object graph.
- CSV/JSON: $O(n)$to store all resident records.

Decision:

CSV or JSON is more suitable when data needs to be shared with other applications or when fields may change frequently. Java serialization is useful when saving and restoring Java object structures, but it requires careful version management.

Sensitive information such as passwords should not be stored in plain-text CSV or JSON.

TASK A5–DESIGN FOR CORRECTNESS AND SAFETY

A5.1 – JDBC Transaction: Transfer ₹500

Theory:

A transaction is a group of database operations treated as a single unit. It follows the ACID properties. If all operations succeed, the transaction is committed. If any operation fails, the transaction is rolled back.

Pseudocode:

START

Connect to database

Disable auto-commit

TRY

Prepare SQL:

UPDATE hostel_wallet

SET balance = balance - 500

WHERE student_id = ? AND balance >= 500

Execute update

If affected rows != 1:

Throw error

Prepare SQL:

UPDATE canteen_wallet

SET balance = balance + 500

WHERE wallet_id = ?

Execute update

If affected rows != 1:

Throw error

COMMIT transaction

Display "Transfer successful"

CATCH error

ROLLBACK transaction

Display "Transfer failed"

FINALLY

Restore auto-commit

Close database connection

END

Explanation:

- setAutoCommit(false) starts manual transaction control.
- commit() permanently saves both wallet updates.
- rollback() cancels both updates if an error occurs.
- The balance condition prevents transferring money when the hostel wallet has insufficient funds.

A5.2 – Preventing SQL Injection

Theory:

SQL injection is an attack in which a user enters SQL commands as input to manipulate a database query.

1. Unsafe SQL query using concatenation

String sql = "SELECT * FROM users WHERE id = '"

+ id + "' AND pwd = '" + pwd + "'";

An attacker may enter:

ID: admin' --

Password: anything

The generated SQL becomes:

SELECT * FROM users WHERE id = 'admin' -- ' AND pwd = 'anything'

The -- comments out the remaining SQL, potentially allowing login without a valid password.

2. Safe SQL query using PreparedStatement

String sql = "SELECT * FROM users WHERE id = ? AND pwd = ?";

```
PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, id);

ps.setString(2, pwd);

ResultSet rs = ps.executeQuery();

if (rs.next()) {

   System.out.println("Login successful");

} else {

   System.out.println("Invalid login");

}
```

Explanation:

- ? represents a parameter.
- setString() safely supplies user input.
- User input is treated as data, not as SQL commands.
- Therefore, SQL injection is prevented.

Note: Real applications should store salted, securely hashed passwords rather than plaintext passwords.

Comparison: Separate Statements vs Transaction

Separate auto-commit statements	**Single transaction**
Each update commits separately.	Both updates are treated as one unit.
Failure may cause an incomplete transfer.	Failure causes rollback.
Data inconsistency may occur.	Maintains data consistency.
Less transaction overhead.	Small additional commit/locking overhead.

Result: The JDBC transaction design ensures that both wallet updates succeed or fail together. PreparedStatement protects login queries from SQL injection attacks. Thus, database correctness and security are improved.

PART B–IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

TASK B1–IMPLEMENT AND INSTRUMENT THE FILE STORE

1. Aim

To implement a bus-pass file storage system using Java file handling and buffered streams. To perform add, search, and route-based listing operations and compare buffered and unbuffered file-reading performance.

2. Theory

A file store saves bus-pass records in a text file. Each record contains five fields:

1. Pass ID
2. Student register number
3. Student name
4. Route number
5. Amount paid

Example:

P1001,192311001,Arun,5,2500.0

P1002,192311002,Meena,7,3000.0

P1003,192311003,Divya,7,3800.0

- Add: Append a new record to the file.
- Find: Search for a pass using its ID.
- List by route: Display all passes belonging to a particular route.
- Buffered reading: Reads data in blocks, reducing expensive physical file-read operations.
- Damaged records: Invalid records are skipped so that searching can continue.

3. Java Code

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
public class PassStore {
    static String fileName = "passes.txt";
    // Check whether a record contains valid data
    static boolean isValid(String[] data) {
        if (data.length != 5) {
            return false;
        }
        try {
            Integer.parseInt(data[3]);
            Double.parseDouble(data[4]);
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }
    // Add a bus-pass record
    static void addPass(String record) throws IOException {
        String[] data = record.split(",", -1);
        if (!isValid(data)) {
            System.out.println("Invalid pass record");
```

```
        return;
    }
    Files.write(
        Paths.get(fileName),
        (record + System.lineSeparator()).getBytes(),
        StandardOpenOption.CREATE,
        StandardOpenOption.APPEND
    );
    System.out.println("Pass added successfully");
}
// Find a pass using its ID
static String[] findById(String id) throws IOException {
    try (BufferedReader br =
             new BufferedReader(new FileReader(fileName))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] data = line.split(",", -1);
            if (!isValid(data)) {
                continue;
            }
            if (data[0].equals(id)) {
                return data;
            }
        }
```

```
    }
    return null;
}
// List all passes for a particular route
static void listByRoute(int route) throws IOException {
    try (BufferedReader br =
            new BufferedReader(new FileReader(fileName))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] data = line.split(",", -1);
            if (!isValid(data)) {
                continue;
            }
            if (Integer.parseInt(data[3]) == route) {
                System.out.println(Arrays.toString(data));
            }
        }
    }
}
// Count lines using buffered reading
static long countBuffered() throws IOException {
    long count = 0;
    try (BufferedReader br =
            new BufferedReader(new FileReader(fileName))) {
```

```
        while (br.readLine() != null) {
            count++;
        }
    }
    return count;
}
// Count lines using byte-by-byte reading
static long countUnbuffered() throws IOException {
    long count = 0;
    int ch;
    int last = -1;
    try (FileInputStream fis =
            new FileInputStream(fileName)) {
        while ((ch = fis.read()) != -1) {
            if (ch == '\n') {
                count++;
            }
            last = ch;
        }
    }
    // Count the final line if it has no newline
    if (last != -1 && last != '\n') {
        count++;
    }
```

```
    return count;
}
public static void main(String[] args) throws IOException {
    // Create sample file if it does not exist
    if (!Files.exists(Paths.get(fileName))) {
        Files.write(
            Paths.get(fileName),
            Arrays.asList(
                "P1001,192311001,Arun,5,2500.0",
                "P1002,192311002,Meena,7",
                "P1003,192311003,Divya,7,3800.0"
            )
        );
    }
    System.out.println("Find P1003:");
    System.out.println(Arrays.toString(findById("P1003")));
    System.out.println("\nFind missing ID:");
    System.out.println(Arrays.toString(findById("P9999")));
    System.out.println("\nPasses in Route 7:");
    listByRoute(7);
    System.out.println("\nBuffered line count: " + countBuffered());
    System.out.println("Unbuffered line count: " + countUnbuffered());
    System.out.println("\nAdding a new pass:");
    addPass("P1004,192311004,Ravi,5,2800.0");
```

```
    }
}
```

Experiment 2 – Search performance

Create files containing 1,000, 10,000 and 100,000 bus-pass records. Measure the time taken by findById() to search for a pass near the end of each file.

Number of records	Search time (ms)
1,000	Measure
10,000	Measure
100,000	Measure

Analysis

- Sequential search takes O(n) time for one search.
- Repeated searches take O(q × n), where q is the number of searches.
- A HashMap index takes O(n) time to build and approximately O(1) average lookup time.
- Indexed searching takes approximately O(n + q) average time, but requires additional memory.

TASKB2– ENGINEER AN ADVERSARIAL INPUT

1. Aim

To demonstrate SQL injection attacks using unsafe SQL queries, compare them with secure PreparedStatement queries, and test how a file store handles damaged bus-pass records.

2. Theory

An adversarial input is specially designed input that attempts to make a program behave incorrectly or bypass its security checks.

SQL injection occurs when user input is directly combined with an SQL query. An attacker can enter SQL commands instead of normal data.

Main objectives:

- Demonstrate SQL injection using unsafe login code.
- Prevent SQL injection using PreparedStatement.
- Test damaged bus-pass records.
- Compare input escaping and PreparedStatement.

3. Java Code

```
import java.io.*;
import java.sql.*;
public class SQLInjectionTest {
   // Unsafe login using SQL concatenation
   static boolean unsafeLogin(Connection con,
                              String id, String password)
                              throws SQLException {
      String sql = "SELECT * FROM users WHERE id = '"
                 + id + "' AND pwd = '" + password + "'";
      Statement st = con.createStatement();
      ResultSet rs = st.executeQuery(sql);
      boolean result = rs.next();
      rs.close();
      st.close();
      return result;
   }
   // Safe login using PreparedStatement
```

```
static boolean safeLogin(Connection con,
                         String id, String password)
                         throws SQLException {
    String sql = "SELECT * FROM users WHERE id = ? AND pwd = ?";
    PreparedStatement ps = con.prepareStatement(sql);
    ps.setString(1, id);
    ps.setString(2, password);
    ResultSet rs = ps.executeQuery();
    boolean result = rs.next();
    rs.close();
    ps.close();
    return result;
}
// Check damaged bus-pass records
static void checkPassFile() throws IOException {
    BufferedReader br =
        new BufferedReader(new FileReader("passes.txt"));
    String line;
    while ((line = br.readLine()) != null) {
        String[] data = line.split(",", -1);
        try {
            if (data.length != 5) {
                throw new Exception("Wrong number of fields");
            }
```

```
            Integer.parseInt(data[3]);
            Double.parseDouble(data[4]);
            if (data[0].isEmpty() || data[1].isEmpty()
                    || data[2].isEmpty()) {
                throw new Exception("Empty field");
            }
            System.out.println("Valid record: " + line);
        } catch (Exception e) {
            System.out.println("Damaged record skipped: " + line);
        }
    }
    br.close();
  }
  public static void main(String[] args) throws IOException {
    System.out.println("Adversarial Input Testing");
    checkPassFile();
  }
}
```

TASK B3–STATEMENT vs PREPAREDSTATEMENT vs BATCH

1. Aim

To implement JDBC database operations using Statement, PreparedStatement, and batch processing. To compare their performance, verify inserted records using SELECT, inspect database metadata, and test transaction rollback when an error occurs.

2. Theory

JDBC provides different ways to execute SQL statements:

1. **Statement:** Executes normal SQL queries. Values are generally included directly in the SQL string.
2. **PreparedStatement:** Uses ? placeholders to pass values safely.
3. **Batch processing:** Groups multiple SQL operations and executes them together, reducing communication overhead with the database.
4. **Transaction:** Uses commit() to save changes and rollback() to undo changes if an error occurs.

3. Database Setup

```
CREATE DATABASE assignment5;

USE assignment5;

CREATE TABLE bus_pass (
    pass_id VARCHAR(20) PRIMARY KEY,
    reg_no VARCHAR(20),
    student_name VARCHAR(50),
    route_no INT,
    amount DOUBLE
);
```

4. Java Code

```
import java.sql.*;
import java.util.*;
public class PassLoader {
    static final String URL =
        "jdbc:mysql://localhost:3306/assignment5";
    static final String USER = "root";
    static final String PASSWORD = "your_password";
```

```
static class Pass {
    String id, regNo, name;
    int route;
    double amount;
    Pass(String id, String regNo, String name,
         int route, double amount) {
        this.id = id;
        this.regNo = regNo;
        this.name = name;
        this.route = route;
        this.amount = amount;
    }
}
// Insert using Statement
static void insertStatement(Connection con, Pass p)
        throws SQLException {
    String sql = "INSERT INTO bus_pass VALUES ('"
        + p.id + "','" + p.regNo + "','" + p.name
        + "'," + p.route + "," + p.amount + ")";
    Statement st = con.createStatement();
    st.executeUpdate(sql);
    st.close();
}
// Insert using PreparedStatement
```

```
static void insertPrepared(Connection con, Pass p)
        throws SQLException {
    String sql = "INSERT INTO bus_pass VALUES (?,?,?,?,?)";
    PreparedStatement ps = con.prepareStatement(sql);
    ps.setString(1, p.id);
    ps.setString(2, p.regNo);
    ps.setString(3, p.name);
    ps.setInt(4, p.route);
    ps.setDouble(5, p.amount);
    ps.executeUpdate();
    ps.close();
}
// Batch insertion: execute every 1000 records
static int loadBatch(Connection con, List<Pass> passes)
        throws SQLException {
    String sql = "INSERT INTO bus_pass VALUES (?,?,?,?,?)";
    boolean oldAutoCommit = con.getAutoCommit();
    int inserted = 0;
    try {
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int count = 0;
            for (Pass p : passes) {
                ps.setString(1, p.id);
```

```
            ps.setString(2, p.regNo);
            ps.setString(3, p.name);
            ps.setInt(4, p.route);
            ps.setDouble(5, p.amount);
            ps.addBatch();
            count++;
            if (count % 1000 == 0) {
                int[] result = ps.executeBatch();
                inserted += countSuccessful(result);
                ps.clearBatch();
            }
        }
        if (count % 1000 != 0) {
            int[] result = ps.executeBatch();
            inserted += countSuccessful(result);
        }
    }
    con.commit();
    return inserted;
} catch (SQLException e) {
    con.rollback();
    throw e;
} finally {
    con.setAutoCommit(oldAutoCommit);
```

```
    }
}
// Count successful batch results
static int countSuccessful(int[] results) {
    int count = 0;
    for (int result : results) {
        if (result == Statement.SUCCESS_NO_INFO) {
            count++;
        } else if (result > 0) {
            count++;
        }
    }
    return count;
}
// Delete records before each experiment
static void clearTable(Connection con) throws SQLException {
    try (Statement st = con.createStatement()) {
        st.executeUpdate("DELETE FROM bus_pass");
    }
}
// Generate sample records
static List<Pass> generatePasses(int n) {
    List<Pass> list = new ArrayList<>();
```

```
    for (int i = 1; i <= n; i++) {
        list.add(new Pass(
            "P" + i,
            "19231" + String.format("%04d", i),
            "Student" + i,
            (i % 10) + 1,
            2500.0
        ));
    }
    return list;
}
// Measure Statement insertion
static long testStatement(Connection con, List<Pass> list)
        throws SQLException {
    clearTable(con);
    long start = System.nanoTime();
    for (Pass p : list) {
        insertStatement(con, p);
    }
    return (System.nanoTime() - start) / 1_000_000;
}
// Measure PreparedStatement insertion
static long testPrepared(Connection con, List<Pass> list)
        throws SQLException {
```

```
    clearTable(con);
    long start = System.nanoTime();
    for (Pass p : list) {
        insertPrepared(con, p);
    }
    return (System.nanoTime() - start) / 1_000_000;
}
// Measure batch insertion
static long testBatch(Connection con, List<Pass> list)
        throws SQLException {
    clearTable(con);
    long start = System.nanoTime();
    loadBatch(con, list);
    return (System.nanoTime() - start) / 1_000_000;
}
// Verify rows and display metadata
static void verifyAndShowMetadata(Connection con)
        throws SQLException {
    String sql = "SELECT * FROM bus_pass";
    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        ResultSetMetaData md = rs.getMetaData();
        System.out.println("\nTable columns:");
        for (int i = 1; i <= md.getColumnCount(); i++) {
```

```
            System.out.println(
                md.getColumnName(i) + " - "
                + md.getColumnTypeName(i)
            );
        }
        int count = 0;
        while (rs.next()) {
            count++;
        }
        System.out.println("Total records: " + count);
    }
    DatabaseMetaData db = con.getMetaData();
    System.out.println("Database: " + db.getDatabaseProductName());
    System.out.println("Driver: " + db.getDriverName());
    System.out.println("Driver version: " + db.getDriverVersion());
}
// Test rollback using a duplicate ID
static void testRollback(Connection con) throws SQLException {
    clearTable(con);
    List<Pass> test = new ArrayList<>();
    test.add(new Pass("P1", "R1", "Arun", 5, 2500));
    test.add(new Pass("P2", "R2", "Meena", 7, 3000));
    test.add(new Pass("P1", "R3", "Divya", 8, 3500));
    System.out.println("\nTesting rollback:");
```

```
    try {
        loadBatch(con, test);
        System.out.println("All records inserted");
    } catch (SQLException e) {
        System.out.println("Error occurred: " + e.getMessage());
        System.out.println("Transaction rolled back");
    }
    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(
             "SELECT COUNT(*) FROM bus_pass")) {
        if (rs.next()) {
            System.out.println(
                "Records after rollback: " + rs.getInt(1)
            );
        }
    }
}
public static void main(String[] args) {
    try (Connection con =
             DriverManager.getConnection(URL, USER, PASSWORD)) {
        System.out.println("JDBC connection successful");
        int[] sizes = {1000, 10000, 100000};
        for (int n : sizes) {
            List<Pass> data = generatePasses(n);
```

```
            System.out.println("\nRecords: " + n);
            System.out.println(
               "Statement: " + testStatement(con, data) + " ms"
            );
            System.out.println(
               "PreparedStatement: "
               + testPrepared(con, data) + " ms"
            );
            System.out.println(
               "Batch: " + testBatch(con, data) + " ms"
            );
         }
         verifyAndShowMetadata(con);
         testRollback(con);
      } catch (SQLException e) {
         System.out.println("Database error: " + e.getMessage());
      }
   }
}
```

TASKB4–FINAL ENGINEERING DECISION

1. Aim

To design a reliable migration process for transferring bus-pass records from a text file into a database using JDBC batch processing and transactions. To compare file storage, object serialization, and database storage for a multi-campus bus-pass management system.

2. Theory

Data migration is the process of transferring information from one storage system to another.

In this task, existing bus-pass records stored in a text file are transferred into a MySQL database.

The migration process should:

1. Read records from the source file.
2. Validate each record.
3. Identify damaged or incomplete records.
4. Insert valid records using JDBC batch processing.
5. Commit successful transactions.
6. Roll back failed transactions.
7. Generate a migration report.

Java Pseudocode

```
START

Connect to MySQL database

Open bus-pass text file

Begin database transaction

FOR each line in file
    Split line into fields
    IF field count is not 5
        Record validation error
        CONTINUE
    END IF
    Validate pass ID
    Validate register number
    Validate route number
```

Validate amount

IF any field is invalid

Record validation error

CONTINUE

END IF

Add valid record to PreparedStatement batch

IF batch size reaches 1000

Execute batch

Clear batch

END IF

END FOR

Execute remaining batch

COMMIT transaction

Generate migration report

CATCH database error

ROLLBACK transaction

Report migration failure

CLOSE resources

STOP

Database Metadata

JDBC provides DatabaseMetaData to obtain information about the database and its tables.

Example:

DatabaseMetaData db = con.getMetaData();

```
System.out.println("Database Name:   + db.getDatabaseProductName());
System.out.println("Database Version: “ + db.getDatabaseProductVersion());
System.out.println("Driver Name: "+ db.getDriverName());
System.out.println("Driver Version: "
      + db.getDriverVersion());
ResultSet tables = db.getTables(
      null, null, "bus_pass", null);
while (tables.next()) {
   System.out.println("Table: "
         + tables.getString("TABLE_NAME"));
}
```